

**MBA
USP
ESALQ**

DATA ENGINEERING III

Prof. Dr. Jeronymo Marcondes

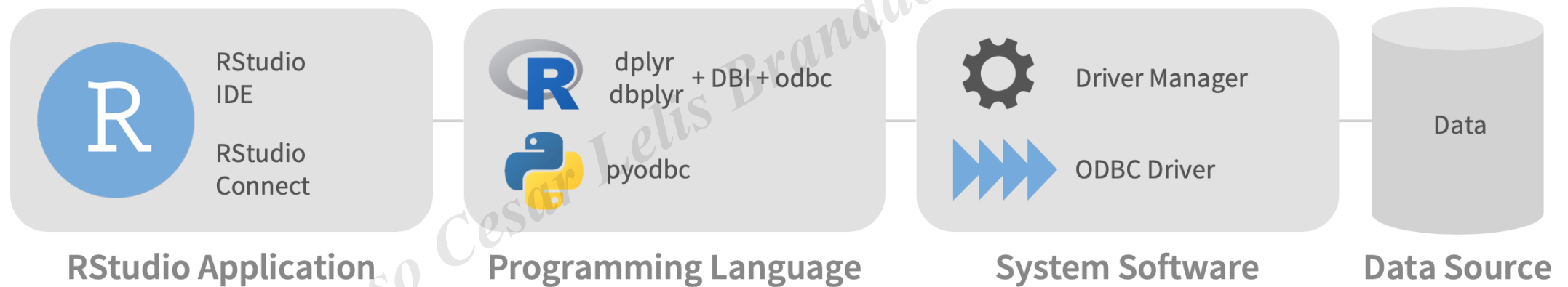
*The responsibility for trustworthiness, originality, and legality of the didactic content presented is responsibility of the professor.

The total or partial reproduction of this material without authorization is **prohibited**. Law No. 9610/98

ODBC and JDBC

- Class focus: ODBC
- Objective: use the capacities of the database to perform our operations.
- Limitations of the languages when working with text files with the necessary data.

Example in R Studio



ODBC and JDBC

- Class approach:
 1. Execute the queries and bring to R.
 2. Use the dplyr to perform the operations within the database.
- All this depends on the understanding of the DBI and DPLYR packages.

DBI package

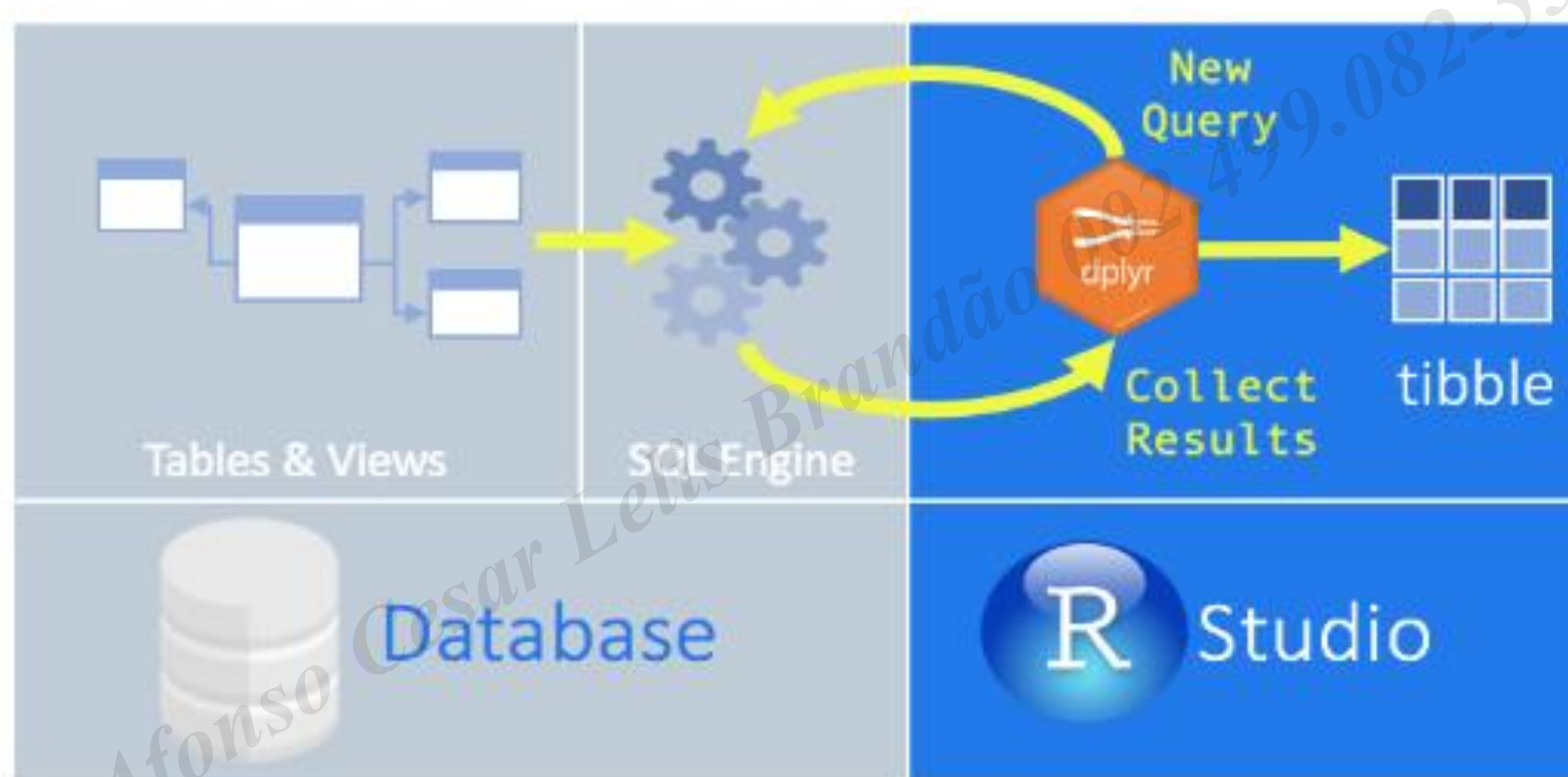
<https://db.rstudio.com/r-packages/dbi/>

DBI

- It allows the division of the connection into a front-end and a back-end.
- Execute query in the database
- It allows performing the interaction between other packages and the database (dplyr)

Commercial databases and open source

DBI



Source: <https://db.rstudio.com/getting-started/>

DBI - structure

```
install.packages("DBI")
```

```
library("DBI")
```

```
con <- DBI::dbConnect(odbc::odbc(),  
                      Driver   = "[your driver's name]",  
                      Server   = "[your server's path]",  
                      UID      = rstudioapi::askForPassword("Database user"),  
                      PWD      = rstudioapi::askForPassword("Database password"),  
                      Port     = 3306)
```

DBI - query execution

```
dbListTables(con)
dbWriteTable(con, "mtcars", mtcars)
dbListTables(con)

dbListFields(con, "mtcars")
dbReadTable(con, "mtcars")
```

DBI - query execution

```
res <- dbSendQuery(con, "SELECT * FROM mtcars WHERE cyl = 4")  
dbFetch(res)  
dbClearResult(res)
```

DBI - best practices

- Close connection after using
- Pay attention to drivers in corporate environment:
<https://db.rstudio.com/best-practices/drivers/>
- Know what is happening when we create a local dataframe with consult.

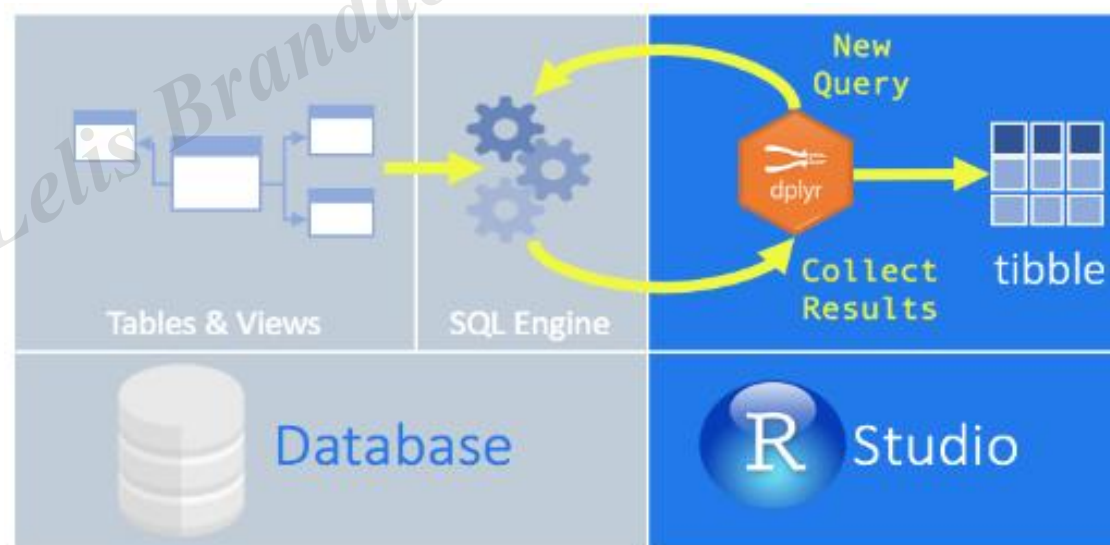
DPLYR

- Main package for Data Wrangling.
- Fast and easy manipulation - it solves more than 90% of the structure problems that can appear.
- It works with dataframes that follow tidydata standards (tibble).

DPLYR

- Dplyr will allow the execution of some wrangling operations directly in the database and use the database computational power without overloading R.

```
install.packages("dplyr")  
library("dplyr")
```



Tibble

- Hadley Wickham:
 1. Each variable is a column
 2. Each observation is a row
 3. Each observational unit is a chart

DPLYR

- The main functions of the dplyr are:

1. `Select()`
2. `Arrange()`
3. `Filter()`
4. `Mutate()`
5. `Group_by()`
6. `Summarise()`

DPLYR - basic structure

- Basic use of the select - select columns of a table

select(dataframe, fields_separated_by_comma)

DPLYR - use of the pipe

- Use of the pipe - more than one dplyr operation in a single table
- The idea of sequential

dataframe %>% select(fields_separated_by_comma)

DPLYR - example of select

```
dim(starwars)
#> [1] 87 14
starwars
#> # A tibble: 87 × 14
#>   name          height  mass hair_color skin_color eye_color birth_year sex
#>   <chr>         <int> <dbl> <chr>      <chr>      <chr>      <dbl> <chr>
#> 1 Luke Skyw...    172    77 blond     fair       blue        19   male
#> 2 C-3PO          167    75 NA        gold       yellow      112  none
#> 3 R2-D2           96    32 NA        white, bl... red        33   none
#> 4 Darth Vad...   202   136 none      white      yellow     41.9 male
#> # ... with 83 more rows, and 6 more variables: gender <chr>,
#> #   homeworld <chr>, species <chr>, films <list>, vehicles <list>,
#> #   starships <list>
```

<https://dplyr.tidyverse.org/articles/dplyr.html>

DPLYR - example of select

```
starwars %>% select(hair_color, skin_color, eye_color)
#> # A tibble: 87 × 3
#>   hair_color skin_color eye_color
#>   <chr>      <chr>      <chr>
#> 1 blond      fair        blue
#> 2 NA         gold        yellow
#> 3 NA         white, blue red
#> 4 none       white       yellow
#> # ... with 83 more rows
```

<https://dplyr.tidyverse.org/articles/dplyr.html>

DPLYR - example of filter

dataframe %>% filter(fields == "certain value")

dataframe %>% filter(fields > "certain value")

dataframe %>% filter(fields < "certain value")

dataframe %>% filter(fields != "certain value")

DPLYR - example of filter

```
filter(starwars, species == "Human")
#> # A tibble: 35 × 14
#>   name      height  mass hair_color skin_color eye_color birth_year sex
#>   <chr>      <int> <dbl> <chr>      <chr>      <chr>      <dbl> <chr>
#> 1 Luke Sky...   172    77 blond      fair       blue        19   male
#> 2 Darth Va...   202   136 none       white      yellow      41.9 male
#> 3 Leia Org...   150    49 brown      light      brown        19   fema...
#> 4 Owen Lars    178   120 brown, gr... light      blue        52   male
#> 5 Beru Whi...   165    75 brown      light      blue        47   fema...
#> 6 Biggs Da...   183    84 black      light      brown        24   male
#> 7 Obi-Wan ...   182    77 auburn, w... fair       blue-gray    57   male
#> 8 Anakin S...   188    84 blond      fair       blue        41.9 male
#> 9 Wilhuff ...   180    NA auburn, g... fair       blue        64   male
#> 10 Han Solo     180    80 brown      fair       brown        29   male
#> # ... with 25 more rows, and 6 more variables: gender <chr>,
#> #   homeworld <chr>, species <chr>, films <list>, vehicles <list>,
#> #   starships <list>
```

<https://dplyr.tidyverse.org/articles/dplyr.html>

DPLYR - example of group_by and summarise

dataframe %>% summarise(new_variable = function applied in variable)

dataframe %>% group_by(variable) %>% summarise(new_variable = function applied in variable)

DPLYR - example of group_by and summarise

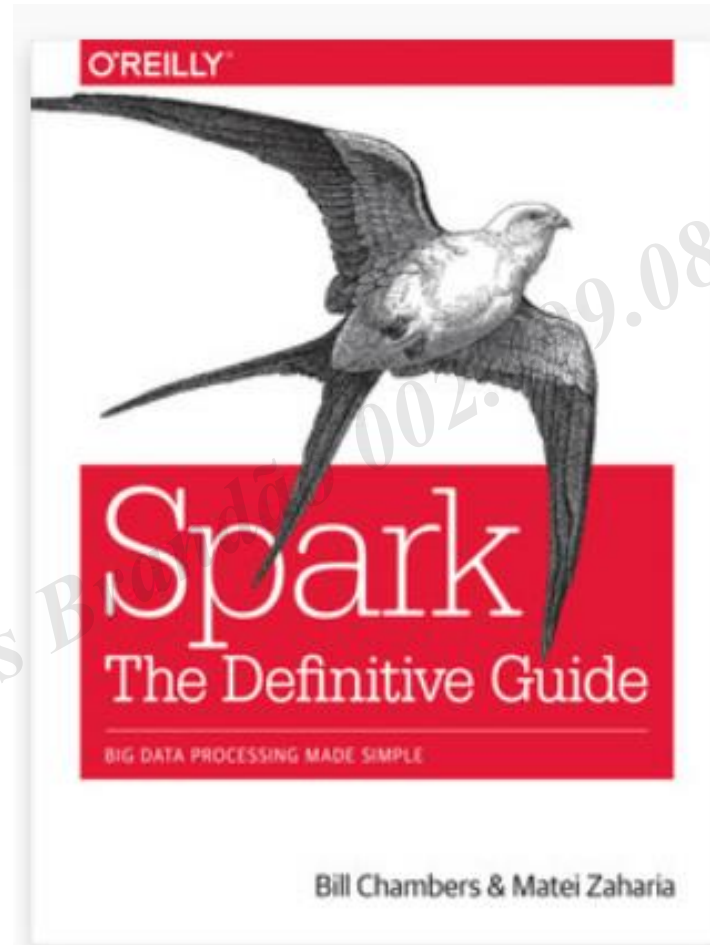
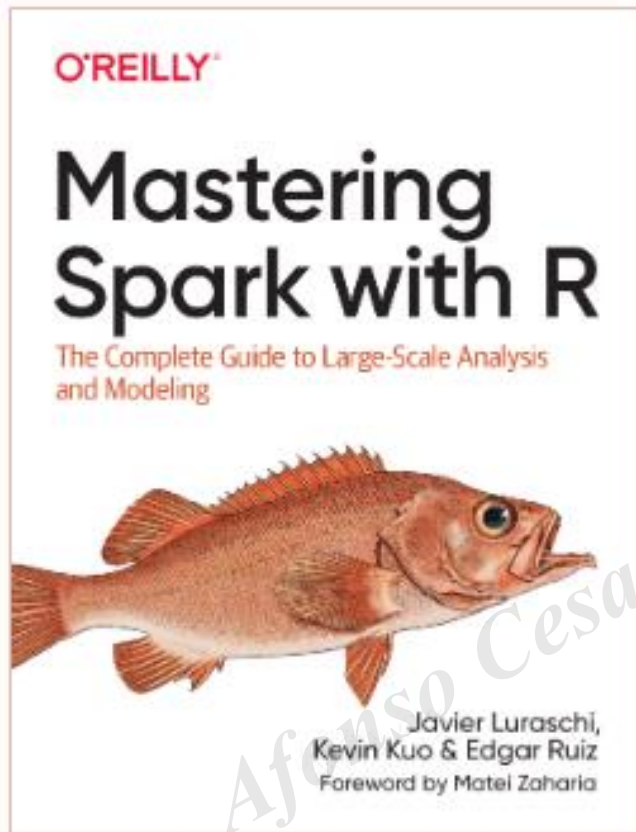
```
starwars %>% summarise(height = mean(height, na.rm = TRUE))  
#> # A tibble: 1 × 1  
#>   height  
#>   <dbl>  
#> 1    174.
```

<https://dplyr.tidyverse.org/articles/dplyr.html>

DPLYR - other cases

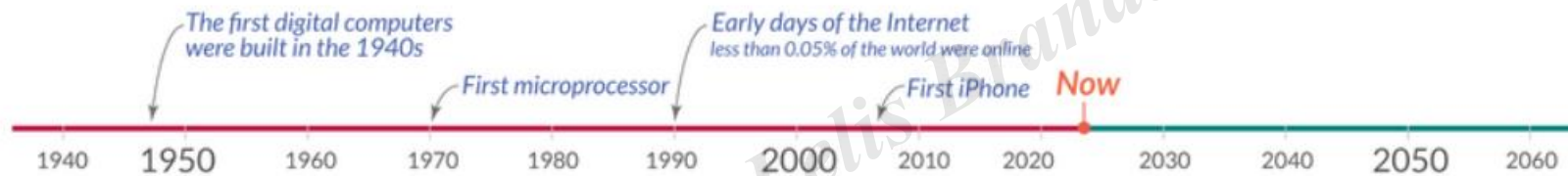
- Amplitude of use of the dplyr
- Class focus

Apache Spark



Big Data

- Constant evolution of computers



Source: <https://ourworldindata.org/brief-history-of-ai>

Big Data

- Computers increasingly more powerful = + data.
- 2005 – speed of the data is too high.
- The parallel computing process begins.

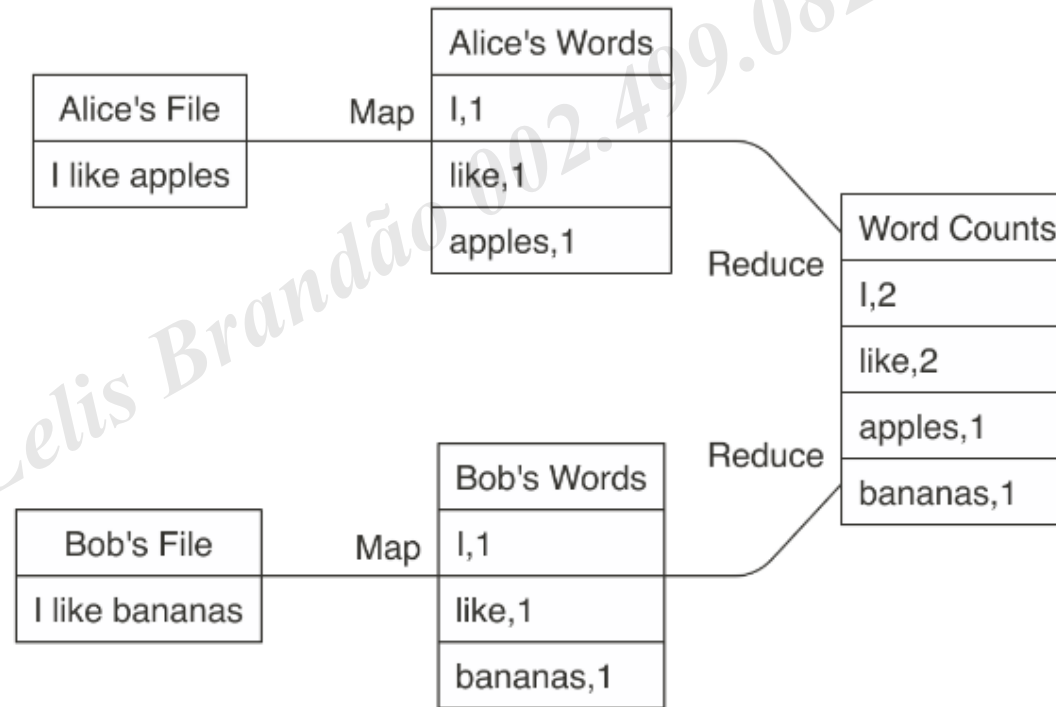
Big Data

- Storage cost also does not reduce.
- Only parallelizing CPU is not enough anymore.
- The Google File System appears

Hadoop

- *Map Reduce*

- *HDFS*



Apache Spark

- UC Berkeley
- "Spark: Cluster Computing with Working Sets".
- Advantages and Disadvantages of the Map Reduce.

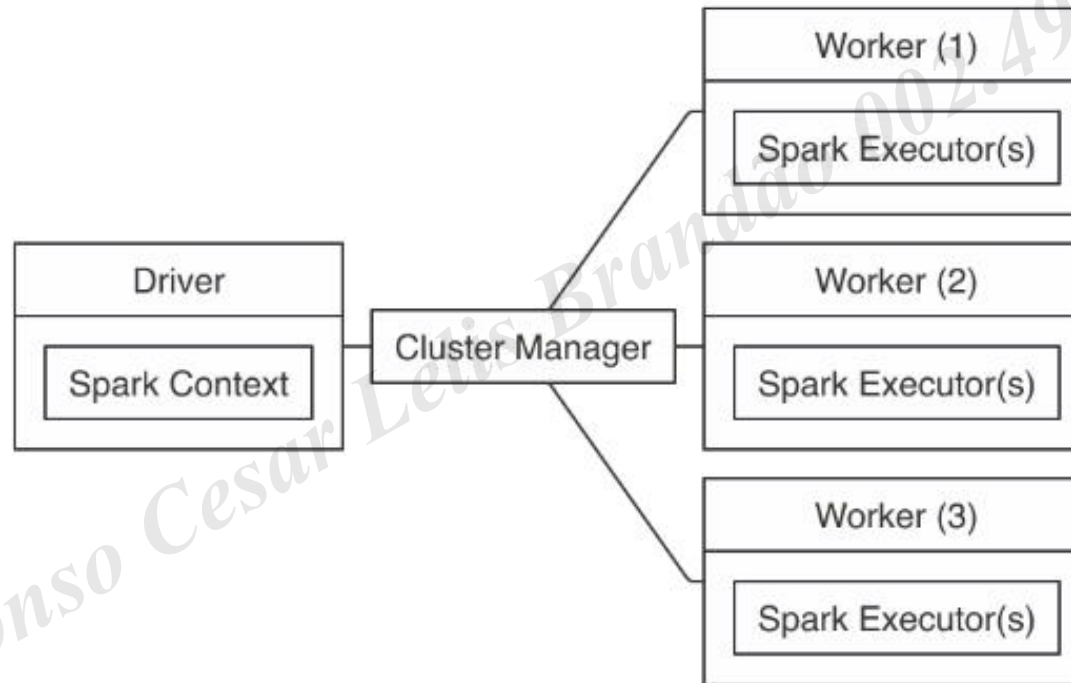
Apache Spark

- Spark appears.
- Possibility of processing "in memory".
- Execution speed.

Apache Spark

- It is a *Engine* - that is, it is used to improve the multiple existing processes. It can be adapted to several languages.
- Structure:
 1. Driver
 2. Cluster Manager
 3. Executors

Apache Spark



Apache Spark

- Cluster Managers: *Spark Standalone*, *YARN*, *Mesos*, and *Kubernetes*.
- Our focus will be in local execution.
- Supported languages: Scala, Python, R, Java, etc.

Apache Spark

- Gateway: Spark Session.
- We do not need to worry about this – interface with R.
- Concept of *sparklyr*

Apache Spark with sparklyr

```
install.packages("sparklyr")
```

```
spark_available_versions()
```

```
spark_install(version = "1.6")
```

Apache Spark

- It is important to highlight in our exercise:

1. We will perform locally.

2. More data = better!

Apache Spark

[colab](#)

MBA
USP
ESALQ

Prof. Dr. Jeronymo Marcondes



<https://www.linkedin.com/in/jeronymo-marcondes-585a26186>